Unobservable effects

Fractional permissions as a discipline for in-place update in a pure language

Gavin Mendel-Gleason
Scidonia

2026

Abstract

A pure functional language can update in place. Lean's `Array.set!` denotes a new array, and its compiled code stores into the existing buffer whenever that buffer has exactly one reference. Both statements are true at once because the store is *unobservable*: nothing in the source language can distinguish a value that was mutated in place from one that was copied, provided the old value had no other reference. That condition, however, is a fact about the compiled code rather than about the meaning of the program, and the compiler decides it per site, from use counts, aliasing and projections. This paper presents a discipline that makes the condition hold where a programmer wants it, and that is erased before it runs. The discipline is a fractional permission protocol in a ghost ledger, packaged in a Hoare monad whose programs can only be built from its own primitives and runs only through one entry point. It is stated once as a fact about compilation to a reference-counted intermediate language, which is where the count is visible, and then lifted: over an arbitrary object, over an arbitrary state type whose laws are proof fields, and, through a description universe in the sense of levitation, over a family of shapes for which one instance suffices. A sealed class in which the library owns the reading, the writing and the encoding of the state closes the last route by which a client can hold an alias. Throughout, the paper separates what is proved in Lean, what is a property of the exported surface, what is measured by running compiled code, and what is trusted of the compiler and the run time.

1 Introduction

Lean's values are immutable. A term such as `a.set! i v` denotes a new array: same length, same elements except at `i`. No term of the language can change a value that already exists, and no program can tell whether some other array happens to share a buffer with this one. The source language is pure, and the compiler is free to make it fast.

The compiler is fast in a specific way. When the array that `set!` is applied to has exactly one reference, the emitted code stores into that array's buffer and returns a pointer to the same buffer. When the array has two or more references, the emitted code allocates a copy and stores into that. The two implementations denote the same value, so nothing has to be decided about which one to use for the program to be correct. The only thing at stake is allocation, and therefore speed. This is the reference-counting discipline of Lean's runtime, described by Ullrich and de Moura [29], with the reuse pass of Perceus [24]: an exact count lets an implementation perform a destructive update precisely when the updated value is about to die.

Both statements — the source denotes a new array, the compiled code writes — are therefore true, and they are compatible because the write is *unobservable*. The observation language is the pure term language, and no term of it can name an address. Unobservability is not a matter of intent, though; it is exactly the condition that no other reference to the buffer is live at the moment of the store. The compiler does not check whether a programmer *wanted* a mutation. It reads the uses of the value off the code it is compiling, and it stores in place when its count is 1.

1.1 The problem

That puts the programmer in an awkward position. Whether a particular write updates in place depends on facts about the *compiled* code: how many times a variable is used in an expression, whether a projection is held across a call, whether an alias is still live, whether a value was produced at run time or is a compile-time constant. None of these are visible in the meaning of the program, and none of them can be inferred from its type. A program can be correct and slow, and a small edit that looks semantically neutral — returning `a.size` alongside `a.set! i v` — can cost a full copy of the buffer. We measured this: on the toolchain used here, `(a.set! i v, a.size)` copies, while `a.set! i v` alone does not.

The engineering goal is therefore not to make the compiler mutate, which is not in our gift, but to arrange programs so that the condition under which it mutates holds where we want the mutation, and to know that it holds.

1.2 The idea

We give the condition a type discipline.

A write needs exclusive access to the object. Reads may be shared. That is the shape of a *fractional permission* [3]: the whole permission, 1, is required to write, while any positive share may be lent to a reader; shares split and recombine, and once every read has been returned the shares sum to 1 and writing is permitted again.

Lean has no linear types. A token value can be copied like any other value, so the permission cannot live in the token. It lives instead in a *ghost ledger* of entries (id, q) that appears only in the types and proofs of the program, and that the compiler erases. A token is a name for an entry; a copied token is a second name for one permission, not a second permission. The program type is a Hoare monad whose precondition and postcondition speak about the ledger and about the state, and the only entry point demands a proof that the program ends holding the whole object.

The discipline is stated once, in a form that does not depend on the object: an object supplies its read and write operations, and the permission rules are uniform over every object. Then it is lifted in three steps.

1. *Any object.* An object joins the interface by naming its read and write operations and by supplying a check that the runtime runs before an update. The check is the identity in the logic, so no specification or proof mentions it, and its compiled form is the exclusivity test.

2. *Any state type.* A second interface makes the logical part of the rules into proof fields of a class. An instance must prove that its initial states are admissible, that writes preserve admissibility, that its encoding of read results is injective, and that a read of an admissible state names no protected location. Every theorem of the generic model is then stated once, for every instance.

3. *Any shape.* The last step is the one that answers "can this be done for shapes we have not written yet?". It can, by *levitation* [4]: describe the shapes in a universe of descriptions, interpret a description as a type, and prove the laws once for the whole universe rather than once per shape. A user's type enters by a bijection to a described type, along which the one instance's operations are transported. When the relation to the described type is an equality rather than a bijection, something stronger becomes possible: the library can own the reading, the writing and the encoding of the state, and a client can then neither introduce an alias nor choose an encoding that hides one.

The chain of the paper is therefore: a pure language whose implementation has *unobservable* effects; a proof that the effects are unobservable, conditional on a reference count of one; a type discipline that establishes the condition; generalisation of that discipline from one array to an

arbitrary object, to an arbitrary state type, and to a family of shapes by levitation; and a sealed surface where the library, rather than the client, decides what a read means.

1.3 Contributions

1. A rendering of the reference-counted write as a rule of a small intermediate language λRC, and a proof that the rule is equivalent to the pure copy exactly when the count is one (section 3). Unobservability is thus made into a statement with a premise, and the premise is the thing a discipline has to deliver.
2. A permission discipline with no linear types in the host language (section 4): fractional permissions in an erased ghost ledger, a Hoare monad sealed by private constructors and a private capability, and an identification of what it does and does not establish — authority is not reference.
3. Generalisation over objects and state types (sections 5 and 7), with the obligations that are proof fields and the negative controls that show they are not vacuous.
4. The levitation step (sections 8 and 9): one lawful instance for every description of a small universe, transport along a bijection for a user's type, and a sealed class that removes the author's choice of read, write and encoding altogether.
5. A worked path calculus over layouts, with take-out writes, and two exclusivity checks that are the difference between an update in place and a copy (section 9).
6. An explicit statement of the correspondence chain that connects the discipline to the compiled program, with each link labelled as proved, structural, measured or trusted (section 10).

All of the code discussed here is Lean 4, on a pinned toolchain, in a library that builds and whose checks are theorems rather than test assertions. The paper quotes the library's declarations verbatim in appendix A; the quotations are read from the source files by line range at build time, and `check-listings.py` verifies that each quoted range still contains the declaration it is claimed to contain.

1.4 How much is claimed

The paper is careful about the status of each statement, and says so where the statement is made. Four labels are used throughout.

- *Proved*: a theorem of the Lean development. For example, that a returned read gives back exactly its fraction, or that a read of a valid state contains no reference.
- *Structural*: a property of the exported surface, shown by probes from another module that the elaborator refuses. For example, that no public route returns the state a session owns.
- *Measured*: a fact about compiled code on this toolchain, established by running an executable that observes addresses. For example, that a write on a uniquely owned array keeps the buffer's address.
- *Trusted*: what remains outside all of it — the compiler's translation, the runtime's primitives, and the arithmetic of the reference count.

No theorem here certifies that a compiled write updates in place. What is proved is the permission protocol and the shape of the interfaces; what is measured is the in-place behaviour of the compiled code; what connects them is a correspondence chain that section 10 states in full, link by link, with the links that are assumptions named as assumptions.

2 Setting: counts, and why the condition is not a typing fact

2.1 A value, an address, and a count

A Lean value of an immutable type is a fully determined piece of data. Arrays and structures of scalar data are represented at run time as heap objects whose fields are values, and a value that occurs several times in the program may be represented by one object with several references. The runtime keeps an exact count of those references. The count is not part of the value; it is part of the machine, and the source language has no term that can read it.

The count n of an object supports exactly three questions.

- $n = 1$: this reference is the only one. If the code that holds it replaces the object by a new value, nothing can still be holding the old one, so the old object's storage may be overwritten in place. The runtime's `lean_is_exclusive` asks this question.
- $n \geq 2$: something else may still read the object. A write must allocate and copy.
- $n = 0$: the object is a compile-time constant, held in a read-only segment. It can never be exclusive, and a write to it always copies.

A fourth case exists in the runtime and matters for anyone writing a test from the count: a count that has overflowed while single-threaded is parked at a sticky value and reads as "unshared" (`lean_is_unshared`, which is $rc > 0 \vee rc \leq \mathsf{LEAN_RC_STUCK_ST}$) while standing for more references than it can still track. `isUnshared` is strictly weaker than `IsExclusive`, and a write that tested the former would lose those references. Appendix A quotes both predicates from the development, which vendors the Lean repository's own machine-checked model of `lean_inc_ref_n` and `lean_dec_ref` [14] and adds the counter-level facts the write discipline rests on: that an extra reference costs the in-place update, that dropping it restores it, that a thread-shared count is never exclusive, and that a never-freed count never becomes exclusive however the adjustments run.

2.2 What the compiler reads

The count is maintained by insertions the compiler makes while translating to its intermediate representation: an increment wherever a value acquires a second reference, a drop wherever one goes out of scope. Which counts those insertions produce is decided by the *use* of values in the code being compiled. Three signals decide every case we have measured.

- An `inc` before a mutation: the value is used again in the same expression, or an alias is live. `(a.set! i v, a.size)` emits `inc x; Array.set! ■ x ...; Array.size ■ x` — one extra use of the same array, and a copy.
- `isShared` before a whole-value rebuild: the reset/reuse pass rewrites the *incoming* cell of a constructor when that cell is exclusive, and allocates when it is not. The pass matches on shape, so when two same-shaped constructors appear in one expression the reused cell can go to the wrong one.
- Take-out against a borrowed read: an element exchanged out of an array is owned by the caller, while an element read by a borrowed accessor is not, and the two differ by exactly one `inc`. Identical intent, opposite outcome.

None of this is visible in the type of the program or in its meaning as a pure term. It is a local, syntactic property of the code the compiler emits. A source language cannot make the property true by typing alone; it can only arrange that the code it accepts is the code whose uses are forgiving.

We take from this a precise formulation of the goal. Give the source language a discipline under which a well-disciplined program compiles to code in which the object at each mutation site has no other reference. The discipline must be checked where the program is written, must cost nothing at run time, and must not depend on the shape of the object.

3 λPure and λRC: the store, and when it is invisible

This section fixes the language in which the claim of unobservability is stated. λPure is the pure source language, and its meaning is what a client is entitled to observe. λRC is the language the compiler targets, with addresses and reference counts and a store instruction. Erasing the addresses of a λRC configuration recovers a λPure value, and a store step preserves that erasure. The condition under which it does is precisely that the stored-into object has one reference, which is the premise a discipline must deliver.

3.1 The pure language

Definition 3.1 (λPure, syntax)**.** Types and terms are given by

$$\begin{array}{rcl} \tau & ::= & \mathbf{1} \mid \mathbb{N} \mid \mathbb{B} \mid \tau \times \tau \mid \tau + \tau \mid [\tau] \\ e & ::= & x \mid c \mid \langle\rangle \mid \langle e, e\rangle \mid \mathsf{inl}\, e \mid \mathsf{inr}\, e \mid \mathsf{case}\ e\ \mathsf{of}\ \{\dots\} \\ & & \mid \lambda x{:}\tau.\, e \mid e\, e \mid \mathsf{arr}(e_1, \dots, e_k) \mid e[e] \mid |e| \mid \mathsf{let}\ x = e\ \mathsf{in}\ e \end{array}$$

The type $[\tau]$ is that of arrays, written `Array` τ in the development. Values are the customary closed forms: constants, the unit, pairs, injections, abstractions, and arrays, where an array value a is a finite map from indices to values.

Definition 3.2 (the pure write)**.** If a is an array value, i an index and v a value, then $a[i \leftarrow v]$ is the array agreeing with a at every index except i, where it is v, if i is within bounds; and a itself otherwise. There is no case in which a changes.

Evaluation is the usual call-by-value big-step relation $e \Downarrow v$ for closed terms, with definition 3.2 as the meaning of the store syntax. Every closed well-typed term evaluates to a value; there is no store, no allocation and no failure in the language. It follows that the observation a client is entitled to make of a run is a value of the pure language, and two runs that produce the same value are indistinguishable to every client context.

3.2 The counted language

λRC has the same terms, and additionally addresses and two bookkeeping instructions.

Definition 3.3 (λRC, syntax and store)**.** Values are extended by addresses $\ell \in \mathsf{Loc}$. A store σ is a finite map from addresses to objects, where an object is a box $\langle v_1, \dots, v_k\rangle$ of values together with a count $n \in \mathbb{Z}$. A configuration is a pair $\langle \sigma, e\rangle$ with e a closed term. The two additions to the syntax are $\mathsf{inc}\ \ell$ and $\mathsf{dec}\ \ell$; $\mathsf{arr}(v_1, \dots, v_k)$ is now an allocating form.

The step relation is call-by-value with evaluation contexts C, and the following rules, omitting the congruence rules.

$$\frac{\ell \notin \mathrm{dom}(\sigma)}{\langle \sigma, C[\mathsf{arr}(v_1, \ldots, v_k)] \rangle \longrightarrow \langle \sigma[\ell \mapsto (\langle v_1, \ldots, v_k \rangle, 1)], C[\ell] \rangle} \text{ ALLOC}$$

$$\frac{\sigma(\ell) = (b, n)}{\langle \sigma, C[\mathsf{inc}\ \ell] \rangle \longrightarrow \langle \sigma[\ell \mapsto (b, n{+}1)], C[\langle\rangle] \rangle} \text{ INC} \qquad \frac{\sigma(\ell) = (b, n) \quad n \geq 2}{\langle \sigma, C[\mathsf{dec}\ \ell] \rangle \longrightarrow \langle \sigma[\ell \mapsto (b, n{-}1)], C[\langle\rangle] \rangle} \text{ DEC}$$

$$\frac{\sigma(\ell) = (b, 1)}{\langle \sigma, C[\mathsf{dec}\ \ell] \rangle \longrightarrow \langle \sigma \setminus \ell, C[\langle\rangle] \rangle} \text{ DEC-FREE}$$

and, for the array write, two rules that differ only in the count.

$$\frac{\sigma(\ell) = (\langle v_1, \ldots, v_k \rangle, 1) \quad i \leq k}{\langle \sigma, C[\ell[i] \leftarrow v] \rangle \longrightarrow \langle \sigma[\ell \mapsto (\langle v_1, \ldots, v_{i-1}, v, v_{i+1}, \ldots, v_k \rangle, 1)], C[\ell] \rangle} \text{ STORE-1}$$

$$\frac{\sigma(\ell) = (\langle v_1, \ldots, v_k \rangle, n) \quad n \geq 2 \quad i \leq k \quad \ell' \notin \mathrm{dom}(\sigma)}{\langle \sigma, C[\ell[i] \leftarrow v] \rangle \longrightarrow \langle \sigma[\ell \mapsto (\langle v_1, \ldots, v_k \rangle, n{-}1), \ell' \mapsto (\langle v_1, \ldots, v_{i-1}, v, v_{i+1}, \ldots, v_k \rangle, 1)], C[\ell'] \rangle} \text{ STORE-}n$$

Definition 3.3 omits a read rule. The one that matters here is the access that *retains* what it reads: an array read whose result is a box must inc that box, otherwise the count would not account for the second reference. Which accesses retain is again the compiler's decision, taken from the uses of the result, and it is the same question as the STORE rules' count. We therefore state reads as reading a scalar leaf without a count change, and note where a retaining read would be an INC in the same style.

3.3 Erasure, and the theorem about the store

Definition 3.4 (erasure)**.** The erasure $\lfloor \sigma \rfloor$ of a store is the partial function from addresses to pure array values that unfolds each object's contents, replacing addresses by the arrays they denote. A configuration $\langle \sigma, e \rangle$ erases to the λPure term obtained from e by replacing each address with the value it denotes under $\lfloor \sigma \rfloor$. The unfolding terminates because the language has no way to build cyclic data.

Write $\langle \sigma, e \rangle \approx \langle \sigma', e' \rangle$ when the two configurations have the same erasure and the same free variables. Since the only observation a client can make is the value of a closed program, and that value is a function of the erasure, $\approx$ implies indistinguishability: no λPure context separates two $\approx$-related configurations.

Lemma 3.5 (bookkeeping is invisible)**.** *The rules* ALLOC, INC, DEC *and* DEC-FREE *preserve* $\approx$.

Proof sketch. ALLOC introduces an address that is not yet reachable from e, so the erasure of the term is unchanged; when the new address is later read, it denotes the box that was written into it, which is the box the pure term denoted. INC and DEC change only counts, and the erasure of definition 3.4 reads objects' contents and never their counts. DEC-FREE removes an object whose count was 1; the address occurred exactly once in the term being evaluated, in the dec, and erasure replaces addresses by denotations, so an address occurring nowhere contributes nothing. Each case is by induction on the evaluation context. □

Theorem 3.6 (the store is invisible exactly at count one)**.** *Let* $\langle \sigma, C[\ell[i] \leftarrow v] \rangle$ *be a configuration, with* $\sigma(\ell) = (b, n)$ *and* i *in bounds. Write* c_1 *for the result of a* STORE-1 *step and* c_2 *for the result of a* STORE-n *step taken in its place. Then* $c_1 \approx c_2$ *if and only if* $n = 1$.

Proof. The forward direction is the interesting one. Suppose $n \geq 2$. Then there is another occurrence of ℓ in the term, since the count exceeds the reference held by the store site. That occurrence is not inside v, whose erasure is fixed by v alone; it is elsewhere in C. After STORE-1, the address ℓ still denotes the updated box at that other occurrence, while after STORE-n it denotes the box as it was, and the updated box lives at the fresh address ℓ'. The two erasures differ at the other occurrence, so they are not $\approx$-related. For $n = 1$ there is no such other occurrence, so the erasure after STORE-1 is the erasure of the term with the updated box at ℓ; which is also the erasure after STORE-n, and lemma 3.5 covers the rest. □

Corollary 3.7 (unobservability)**.** *If a program's* λRC *execution reaches every store with count one, then its result is a value of* λPure*, and the choice between storing in place and copying at each of those sites is invisible to every pure context.*

Theorem 3.6 says what the discipline has to achieve, and says it without mentioning types: the count at the store, and nothing else. It also explains why a discipline is needed at all rather than an assumption. The count is maintained by the compiler from the uses of values; the source program's meaning does not determine it, and no amount of care in the *meaning* of the program will move it. What a discipline can do is constrain the code so that the uses are the forgiving ones. That is what the next section builds.

Remark 3.8. Theorem 3.6 is a theorem about λRC as defined here, not about Lean's implementation. The implementation is the subject of section 10, where the correspondences between this calculus, the emitted code and the runtime are either measured or named as assumptions.

4 A permission discipline, erased

The discipline has to establish the premise of theorem 3.6 without measuring anything at run time, without a linear type system in the host language, and without restricting which objects it applies to. This section gives the core of it: fractions in a ghost ledger, a program type that speaks about the ledger, and a sealing discipline that keeps a client from obtaining a program except through the primitives.

4.1 Fractions, held fractions, and the ledger

Definition 4.1 (permission)**.** A *fractional permission* is a rational q with $0 < q \leq 1$. A write requires the whole permission; a read requires a positive share. Shares lend and return additively.

The reading we use is Boyland's [3]: a permission is a share of an object, so authorisation to write is the whole share and authorisation to read is any part of it. What the discipline has to prove is that the shares that are out at the moment of a write amount to nothing.

Definition 4.2 (ledger and held fraction)**.** A *ledger* is a list of pairs (id, q), one for each read that is currently lent out. The *held* fraction of a ledger is

$$\mathsf{held}(l) \;=\; 1 - \sum_{(n,q)\in l} q.$$

Two facts about definition 4.2 are what the protocol rests on: lending a fraction lowers the held fraction by exactly that fraction, and returning an entry raises it by exactly the same fraction. The latter is `Ledger.held_erase_of_mem`, quoted in appendix A: for any $(n, q) \in l$, $\mathsf{held}(l.\mathit{erase}\,(n, q)) = \mathsf{held}(l) + q$. The ledger is thus a complete account of how much of the object is unavailable to a writer, and the sum returns to its starting point once every entry is erased.

4.2 Why the permission cannot live in the token

The obvious design puts the permission in the value that represents it: a token of some linear type, consumed by a release. That design does not survive contact with Lean, and the reason shapes everything that follows.

Lean has no linear, affine or uniqueness typing. A value can always be copied. If the permission were a token carrying the fraction, then `let t' := t` would produce a second permission out of one, and nothing in the type system would notice. This is not a limitation of our particular interface; it is a property of the host language, and it rules out the direct translation of any ownership discipline that assumes linear values [1, 2].

The permission therefore lives outside the values, in a place the language can reason about but the program cannot inspect: the type of the program, and the proofs attached to it. The token becomes a *name* for a ledger entry. A copied token is a second name for the same entry; it is usable only while the entry is present, and a release removes the entry, so a second release of the same token has a false precondition. There is no second permission, only a second handle.

The same argument applies to the ghost identifier. The fraction alone would not say which outstanding read a token names, since two concurrent reads may hold equal fractions, so each entry carries an identifier chosen by the caller. There is no runtime counter for it: the ledger does not exist at run time, so the identifiers must be chosen in the proof rather than drawn from the machine.

4.3 The program type

Definition 4.3 (pre- and postcondition)**.** $\mathsf{Pre}\ \sigma = \mathsf{Ledger} \to \sigma \to \mathsf{Prop}$ and $\mathsf{Post}\ \sigma\ \beta = \mathsf{Ledger} \to \sigma \to \beta \to \mathsf{Ledger} \to \sigma \to \mathsf{Prop}$. Both are predicates over the ghost ledger and the state, and both live in Prop.

Definition 4.4 (program)**.** $\mathsf{HSt}\ \sigma\ P\ \beta\ Q$ is the type of a program over the state σ whose precondition is P and postcondition Q. Its constructor is private to the module that defines it, so a client cannot build a program out of nothing, and its single accessor is private and demands a *capability* Cap, a private proposition that only the module can inhabit and that the compiler erases. The only entry point from outside is

$$\mathsf{exec} : \mathsf{HSt}\ \sigma\ P\ \beta\ Q \to (s : \sigma) \to P\,[\,]\,s \to (\forall b\ l'\ s',\ Q\,[\,]\,s\ b\ l'\ s' \to \mathsf{held}(l') = 1) \to \beta \times \sigma.$$

Two things about definition 4.4 are load-bearing, and both are refinements that were forced by a failure rather than chosen in advance.

The private constructor stops a client from *building* a program. It does not stop a client from *running* one: Lean's generated eliminators for a structure expose its fields, so a private field alone still leaves the accessor callable. Hence the capability. `Cap` is an uninhabited private proposition whose only constructor is in the defining module, so an accessor that demands one cannot be called from outside, and the eliminators cannot supply one either. Both refusals are pinned in the source by checks that the elaborator must reject, so a future change that reopens one fails the build.

The precondition of `exec` is the empty ledger, and its closing obligation is that the program ends holding the whole object. That is the whole of the linearity this design has: not linearity of tokens, which Lean cannot give, but exactness of the ledger, which it can. A program that never returns a read fails the closing obligation; a program that returns one twice fails the precondition of the second release; a program that reads with a token whose entry is gone fails the precondition of the read.

4.4 The primitives

Every primitive carries its specification in its type. The four that matter are the following, quoted verbatim with their bodies in appendix A.

Definition 4.5 (primitives)**.** With σ an object whose instance is in scope, and *held* as in definition 4.2,

$$\begin{aligned}
\texttt{acquire}\ n\ q &: \mathsf{HSt}\ \sigma\ (\lambda l_.\ n \notin \mathrm{ids}(l) \wedge 0 < q \wedge q \leq \mathsf{held}(l))\ (\mathsf{ReadTok}\ n\ q)\ (\lambda l\ s_.\ l' = (n,q) :: l)\\
\texttt{read}\ t\ o &: \mathsf{HSt}\ \sigma\ (\lambda l_.\ (n,q) \in l)\ (\mathsf{Result}\ o)\ (\lambda l\ s\ v\ l'\ s'.\ l' = l \wedge s' = s \wedge v = \texttt{observe}\ o\ s)\\
\texttt{release}\ t &: \mathsf{HSt}\ \sigma\ (\lambda l_.\ (n,q) \in l)\ \mathbf{1}\ (\lambda l\ s_.\ l' = l.\mathit{erase}\,(n,q) \wedge s' = s)\\
\texttt{write}\ o &: \mathsf{HSt}\ \sigma\ (\lambda l_.\ \mathsf{held}(l) = 1)\ \mathbf{1}\ (\lambda l\ s_.\ l' = l \wedge s' = \texttt{update}\ o\ s)
\end{aligned}$$

Sequence is `bind`, whose precondition requires the first program's postcondition to establish the second's precondition, and whose postcondition composes the two existentially. Weakening is `conseq`: a stronger precondition and a weaker postcondition are both sound. `withRead` is `acquire`, the body, `release`, and return the body's result: it is derived from the primitives and adds no trust, and it has the property that the fraction comes back however the body ends, so a client cannot leak it and has no release of its own to forget.

Example 4.6 (two readers and a write)**.** A program that lends a third and two thirds to two readers, reads with each, returns both, and then writes:

```
acquire 0 (1/3)  >>>= fun t1 =>
acquire 1 (2/3)  >>>= fun t2 =>
read t1 0        >>>= fun v  =>
release t1       >>>= fun _  =>
release t2       >>>= fun _  =>
write (0, 9)     >>>= fun _  => ret v
```

Each `acquire` is justified by the held fraction: of the whole, a third is lent and then two thirds, leaving nothing, which is exactly what the write requires. The program is accepted; a second `release t1` would not be.

4.5 What is refused, and how the refusals are stated

Each way of misusing the discipline is refused, and the refusal is not merely a tactic that gave up: it is a theorem that the obligation of the offending program is *false*. Writing while a read is outstanding, returning a token twice, reading after returning, taking out more than is held, reusing an identifier, and never returning a read are each a proposition about `HSt.pre` or about the closing condition of `exec` that the development proves false.

Stating refusals this way matters for two reasons. A check whose reason could be a failure of the elaborator says nothing when the elaborator succeeds later for an unrelated reason; a theorem of falsity says that no proof exists, so the program can never be accepted, on any toolchain. And the theorem names the obligation, so a reader can see which rule of definition 4.5 is doing the work.

4.6 Erasure: what the compiler drops

The discipline is specification. Every part of it that is a type former or a proposition is erased by the compiler, and the erasure is observable in the intermediate representation.

- The ledger, every precondition, every postcondition, every proof and the capability `Cap` are propositions or type formers; they appear as ■ in the emitted `Lean.IR`.
- A read token is a *type*, so it is not erased, but it has no fields: it compiles to a constant scalar, the same representation as `Unit`, and allocates nothing.

- The ghost identifier and the fraction are ordinary `Nat` and `Rat` arguments. Lean erases arguments by their *type* — proofs, types, type formers — and not by whether the body uses them, so these are not erased as such. Written literally they are dropped by the compiler's arity reduction; written symbolically, as `r/2`, they are not evaluated either, because a call to a reduced entry removes the argument code as dead. They survive as unused borrowed parameters only in a definition that is generic in them, and they are computed and boxed only when a call cannot be brought to a reduced entry at all, as through a procedure value.

The result is that a compiled client program threads one reference-counted object and nothing else. The permission accounting costs no allocation on the run-time path, which is the property that makes the design worth the trouble: a discipline that allocated to prove that no allocation happens would be self-defeating.

4.7 Authority is not reference

The discipline establishes an *authority* invariant and not a *reference* invariant, and the gap between them is where the rest of the paper lives.

$\mathsf{held}(l) = 1$ says that no reader holds a share. It is a statement about the ledger. Whether the object is exclusively owned by the machine is a statement about the object graph, and the ledger is ghost state that the object graph knows nothing about. Three routes defeat the inference from authority to reference.

1. The caller's binding. `exec` takes the state from the caller. A caller that still holds its own reference to that state has two references, and the write copies. Measured: the shared control case in the harness.
2. A read result that shares memory. A consumer of a read may return its fraction honestly and keep the value it received. If that value shares mutable structure with the state, the next write copies. The array interface's read returns an `Option` α, which for an array element is a scalar, so the leak does not arise there; it arises as soon as an object's read can return a part of the state.
3. The library's own temporaries. A take-out write returns a pair, and while that pair is live the extracted element has two references. The check must therefore run after the pair has been destructured, which is a fact about the emitted code and was found by measurement, not by reading the source.

So the permission discipline closes one half of the problem: it makes a write's *authority* condition a proof obligation rather than a hope. The other half — that no other reference to the object exists — is a property of the interface and of the code the interface is compiled to, and section 5 begins the work of making it structural.

5 Any object, not only arrays

Nothing in section 4 mentions arrays. This section makes that explicit by letting an object join the interface, and by moving the in-place check into the object, where it can know what a write touches.

5.1 The object interface

Definition 5.1 (object)**.** An object σ is an instance of a class with the following fields: a type family `Result` of the values its read operations yield, a reading function `observe`, a writing function `update`, a check `check : String` $\to$ `WriteOp` $\to \sigma \to \sigma$, and a law `check` *site o s* =

s. The two operation types are class *parameters* rather than fields, because an argument whose expected type is a projection of the class is not reducible until that projection's instance is known, and instance search does not resolve the nested goal; with the types as parameters, a literal such as the array's index, or the pair $(0, 7)$, can be given as an operation.

The check is the identity in the logic. That is what the law says, and it is why no specification or proof mentions it: a specification states what `update` does, and `update` is what runs after the check returns. Its compiled form is something else entirely. The default check is `assertExclusive`, whose logical content is the identity and whose compiled content, supplied by an `implemented_by` attribute, tests the object's exclusivity and, when the process environment asks for it, reports the site and exits. A write therefore pays one out-of-line call, one load and one branch, and nothing on the write path reads the environment: the switch is read once, at initialization.

Definition 5.2 (the array instance)**.** An array joins the interface as follows: a read operation is an index and yields `a[i]?`; a write operation is an index and a value and sets that element; and the check is the array's exclusivity check, which names the array's size in its report.

Example 5.3 (a second object)**.** A structure of a count and a log joins the interface with a check of its own: exclusivity of the structure, and, for the operation that appends to the log, exclusivity of the log. A projection of a constructor takes no reference of its own, so checking the field does not itself share it. This is what the check hook is for: the operation is a parameter, so a check can leave alone what that operation does not write. An abort in checked mode then means that *this* write is about to copy, not merely that some part of the object is shared.

5.2 The two rules that no specification can state

Two obligations attach to any instance, and neither is enforceable from the specifications, because both are about how Lean's values are represented rather than about their mathematics. The interface states them in prose, in the documentation of the class.

1. `observe` must not return a reference to a part of the state that some write updates in place. A read that hands back the array field of a structure leaves an alias behind, and the next write copies that field. Sharing values that no write mutates in place — a `Nat`, or an element's own value — is harmless and intended; it is the mutable parts that a read result must not name.
2. `update` must not keep an old part of the state alive. Writing one field with functional update syntax rebuilds the state around the other fields; a rebuild that copies them makes the next write copy them.

These obligations are the subject of section 7 and section 9, where they become proof fields of a class and then disappear into the library. Before that, the discipline already has a property worth stating: leaking an alias does not make a program wrong. Lean is pure, so a missed in-place update costs an allocation and a copy, and never a wrong result or a race. The discipline is a performance discipline, and its failure mode is time.

6 A notional interface: fractions internal, one runner

The next step removes the fraction from the client's view without removing it from the theory. The client writes effectful-looking code over an object it cannot name; the fractions are what justify the interleavings the semantics admits.

6.1 Readers and programs

The surface has two computation types. A `Reader` may read: `get` at an index, `size`, and `parallel`, which pairs two readers. A `Program` may read and write: `readPhase` runs a reader, and `set` writes an element and reports whether the write was in bounds. A program is run by `runFresh`, which is given a size and a fill value, creates the array itself, runs the program, and returns the result; `runFresh` is an `IO` action precisely because the array must be created at run time, and a compile-time constant can never be exclusive.

Both computation types carry their command tree behind a private constructor and a private field, and the interpreter that runs a tree against an array is private to its module. `runFresh` is therefore the only runner, and no public function of the executor accepts an array. What the types do expose — through generated eliminators and through `Reader.treeOf` and `Program.treeOf` — is a tree of reads, writes, forks and pure continuations over observations: a recipe, with no state in it. The model's own evaluators are public and pure; they take the array from the caller, they are what the theorems are stated about, and they are not a route to the array a session creates.

6.2 The semantics, and why the fractions are still there

The model is a small-step relation over threads, where a thread is a running read phase. It has a step for every interleaving of a fork's branches, and that is what makes `parallel` notional: the shipped executor runs the left branch and then the right, while the model admits every order, and the theorems say the two agree.

- Shares are conserved at a fork and restored at a join, and a fork halves its parent's share. A well-formed leaf's shares sum to the thread's share and stay positive.
- No write step has a source while a read phase runs, and a well-formed reading configuration leaves the writer no authority. This is the semantic counterpart of the ledger's $\mathsf{held} = 1$.
- A join becomes a run only when both children have finished.
- A read's result is a function of its observations alone; a write preserves the array's size and yields `true` exactly when it is in bounds.
- Every interleaving computes what the sequential denotation computes. The harness cases that arm the in-place check for this executor exit zero, which is the run-time counterpart of the same claim.

The fractions are specification, and more so here than in section 4: on the run-time path there is neither a token nor a ledger, and the compiled executor threads one array and the command trees it is walking. A read phase therefore costs no allocation for permissions; only the continuations the trees already carry.

6.3 What has changed about the discipline

Sealing the state changes the role of the fractions, and it is worth being explicit because it changes the pitch.

An object's read and write operations are only reachable through `Reader` and `Program`, and the state those operate on is created by `runFresh` and never returned. A client cannot hold a reference to the session's array, because it never receives one, and no operation returns one. Under that restriction, the authority that the fractions account for and the exclusivity the runtime needs no longer diverge through the client: the *types* give the write its exclusivity — a reader has no write, and a write exists only outside a read phase — and the fractions account for

something else, which is *sharing*: how a fork splits authority, why a join waits for both branches, and why every interleaving agrees with the sequential denotation.

That is a cleaner story than the one section 4 tells, and it also makes the remaining gap precise. Sealing removes the client as a source of aliases. It does not remove the library's own temporaries, it does not touch the compiler, and it does not touch the runtime. Those are the subjects of section 9 and section 10.

Remark 6.1 (why `parallel` is notional)**.** A real concurrent backend would need more than permissions. On this toolchain, an array captured by a `Task.spawn` closure answers `false` to the exclusivity test for good, even after the task has been joined: it is marked as shared between threads, and the next write returns a copy. Getting the array back to a single owner is work the permission discipline does not do. The structured executor is therefore sequential, and the fork/join is a semantics statement rather than a claim about tasks.

7 Any state type: laws as proof fields

Section 5 leaves two obligations in prose: a read must not name the mutable state, and a write must not keep an old part of it alive. This section turns the logical content of the first into proof fields, so that the theorems of the model hold of any instance that proves them, and states exactly how much of the second remains.

7.1 A language in which a reference can be written down

The first obstacle is that the obligation is inexpressible: "a read must not return a reference to the state" is a sentence about values, but the values a read returns are of arbitrary type, and the type does not say whether a reference is inside.

The remedy is to fix a language for read results, small enough that a reference is a syntactic object in it.

Definition 7.1 (values and detachment)**.** The `Value` universe has a constructor for each shape a read may return — a unit, a natural, an unsigned word, a boolean, an optional, a sum, a pair — and one more, `.loc` ℓ, which names a location. `NoRef` ℓ `v` holds of a value v that contains no `.loc` ℓ anywhere inside it, however deeply nested: it descends through `some`, `inl`, `inr` and `pair`, so a location hidden beneath a compound observation is caught.

7.2 The class

Definition 7.2 (lawful state variable)**.** A state variable is an instance of a class with a state type σ, an initializer type, read and write operation types, and: the type of value a read yields; `initial`; a predicate `Valid` naming the states the variable may be in; `observe`, a pure function of an operation and a state; `replace`, which performs an exclusive logical replacement and reports whether it took effect; and `encodeRead`, which translates a read result into a `Value`. Membership is by proof of four laws.

(L1)	`initial_valid`	$\forall i,$ `Valid` (`initial` i)
(L2)	`replace_valid`	$\forall w\ s,$ `Valid` $s \to$ `Valid` (`replace` $w\ s$).1
(L3)	`encodeRead_injective`	$\forall o\ x\ y,$ `encodeRead` $o\ x =$ `encodeRead` $o\ y \to x = y$
(L4)	`observe_detached`	$\forall \ell\ o\ s,$ `Valid` $s \to$ `NoRef` ℓ (`encodeRead` o (`observe` $o\ s$))

Each law closes a specific hole in the generic theory.

- *L1* is what makes a fresh session admissible. The generic theorems assume a well-formed configuration, and a well-formed configuration includes `Valid s`. For a fixed-capacity buffer it fails for an initializer of the wrong capacity; the development carries that counterexample.

- *L2* makes reachability an induction: with L1, every state reachable from a fresh session is admissible, and so is every heap cell after a write. It fails for a write that grows a fixed-capacity buffer.
- *L3* is there to keep L4 from being vacuous. Without it, encoding every result as the unit would satisfy L4 trivially and say nothing. No generic theorem uses L3 directly.
- *L4* is the detachment the write discipline needs: a read of an admissible state names no protected location.

Being lawful is not inherited. An existing object instance is not automatically a lawful state variable, and none of the historical instances is claimed as one. Membership is a proof, and the proofs are in the check library.

7.3 The model, and what it buys

A second model accompanies the class: command trees for readers and programs with their binds; a small-step relation over configurations; a denotation; a replay function, which is the one noncomputable definition in the theory, since comparing two operations is a logical side condition; a well-formedness predicate for reading threads with a share for each leaf; and a safety predicate for command trees.

Stated once for every lawful instance, this model gives the same properties the structured interface gave for one array: share conservation, exclusion, join discipline, write preservation, trace replay and schedule independence. On top of it sits a single-root heap, in which `At` relates a location to the state it holds, `ReadAt` and `WriteAt` describe a read and a write step against it, reads leave it unchanged, a write changes only its own cell, and a read–write–read sequence is a snapshot: the first read keeps the value it took from the *old* state, and the second is the written one.

The array is an instance, and two bridge theorems state that the instance's denotation is the structured executor's: the lifted trees are the trees the interface already exposes, and the runner's equation composes the model's own equation with the program bridge. Thus the general theory does not replace the structured interface; it explains why that interface's theorems hold.

Example 7.3 (the laws are not vacuous)**.** Each law has a countermodel, and each is a theorem that a specific instance's obligation is false, paired with the affirmative control of an instance that satisfies it. A read that returns `.loc` ℓ fails L4; a write that leaves the admissible states fails L2; an encoding that forgets what it was given fails L3. The development proves each.

7.4 What the laws do not say

The laws constrain the *encoding* and not the object graph, and the development carries a witness to that gap.

Example 7.4 (a lawful instance whose read is the state)**.** Take an array of machine words. Encode it as a nested pair of words, and the encoding is injective and contains no `.loc` anywhere, so L3 and L4 hold; take `Valid` to be trivial and `replace` to be the array write, and L1 and L2 hold. All four laws hold. Yet take the read to be the identity, and the read of the state returns the protected array itself.

The instance is legal, and it is exactly what L4's proof obligation cannot catch: Lean's logic sees values, and an alias and a copy of an array are equal values. `NoRef` is a statement about the encoding, so an author who chooses both the encoding and the read can choose them consistently and still hand back the state.

Example 7.4 is the reason the second half of this paper exists. A discipline that lets the instance author decide what a read means has an escape hatch, and the only way to close it is to

take the decision away. There are two ways to take it away, and they differ in how the author's type is related to a type the library knows: by a bijection, in section 8, or by an equality, in section 9.

8 Any shape: a description universe

The lawful-state-variable class is general over types but not over shapes: each instance proves its own laws, and an author who wants a new state type must discharge them all. This section removes that per-shape work with the pattern of levitation [4]: describe the shapes in a universe, interpret a description as a type, and prove the laws once for the whole universe.

8.1 The universe and its interpretation

Definition 8.1 (descriptions)**.** A description is a unit, a natural, or a sum of two descriptions: $\mathsf{Desc} ::= \texttt{.unit} \mid \texttt{.nat} \mid \texttt{.sum}\ d\ d$. The type a description denotes is $\mathsf{El}\ \texttt{.unit} = \mathbf{1}$, $\mathsf{El}\ \texttt{.nat} = \mathbb{N}$, and $\mathsf{El}\ (\texttt{.sum}\ d_1\ d_2) = \mathsf{El}\ d_1 + \mathsf{El}\ d_2$. The definition of `El` is an abbreviation rather than a definition, so for a client it is reducible: a description of a slot has El equal to $\mathbf{1} + \mathbb{N}$ definitionally, and a literal injection can be given at such a type without an ascription.

The universe is deliberately non-recursive. Its types are built from the unit, the natural and the sum alone, so a proof over it is a finite case analysis rather than a fixpoint. Recursion is the obvious next extension and is not attempted here.

Definition 8.2 (one instance)**.** `encodeEl` encodes an element of *any* description as a `Value`, and the facts the class needs of it hold of every description: `encodeEl` is injective, and it names no location. Both are proved once, by a structural induction on the description with one case per constructor, rather than once per description.

Because two descriptions can denote definitionally equal types, the description indexes a wrapper: `Described d` is a type whose instance argument is the description. Then

$$\texttt{describedStateVar}\ d : \mathsf{LawfulStateVar}\ (\texttt{Described}\ d)\ (\mathsf{El}\ d)\ \mathbf{1}\ (\mathsf{El}\ d)$$

is a single instance serving every description, and its four laws are proved once, at the description rather than at the type.

8.2 A user's type, by transport

One instance for the universe is not yet an instance for a type a user wrote. The second half of the pattern carries the instance across a bijection.

Definition 8.3 (transport)**.** Let τ be any type with functions $\mathit{toEl} : \tau \to \mathsf{El}\ d$ and $\mathit{ofEl} : \mathsf{El}\ d \to \tau$ that are inverse in both directions. Then `transport` yields a lawful instance at τ whose operations are the universe instance's conjugated by the isomorphism, and whose every law field is the universe's law after an inverse law cancels the conjugation.

Example 8.4 (the slot, by transport)**.** A slot is a genuine inductive type — empty, or holding a natural payload — described by a unit summed with a natural. Its instance is one line:

```
transport slotDesc Slot encode decode decode_encode encode_decode
```

Every operation is the universe instance's conjugated by `encode` and `decode`, and every law is the universe's, so nothing specific to the slot's description is proved. The theorems of the generic model then apply at it: a read after a write returns the written value; a snapshot survives, with the old value still held by the pre-write heap and the cell holding the new one; and a single root cell changes shape, *empty* $\to$ *full* n $\to$ *empty*.

A second datatype costs only its isomorphism. The development checks a transport for the booleans at the description `.sum .unit .unit`, which the slot does not use, so the transport is not exercised only by its one client.

This is the pattern in miniature, and it is worth naming the three roles, because the remaining sections scale it. There is a *model* — the description universe and its interpretation. There is a *bijection* between the model and the user's type. And there is a *transfer*: observations and updates are stated in the model and proved once there, then carried to the user's type along the bijection, so that the operation the user calls runs on the user's own type while the reason it is correct lives in the model.

8.3 Where the bijection is not enough

A transport makes a user's type lawful, but it leaves the shape of the read and the write in the user's hands: the author supplies `encode`, `decode` and the two inverse laws, and can still choose a read that names the state, as in example 7.4. What transport adds over a hand-written instance is that the laws are not re-proved; it does not add a guarantee about what a read returns.

To get that, the relation between the author's type and the library's type has to be tighter than a bijection. A bijection gives operations that are the model's *up to* conversion, and the conversion is the author's; an equality gives operations that *are* the model's, with the conversions erasing. Which is what the next section builds, and the price is that the library must decide what a read and a write mean, and therefore must have a universe in which they can be defined for every shape at once.

9 Owning the sense of the state

This section gives the library the entire meaning of the state. It has three parts: two universes, one for what a read may return and one for what a state may be; a calculus of paths over the second, with reads that end in the first and writes that take an element out before updating it; and a class whose instances supply nothing but the layout and an equality.

9.1 Two universes, and the asymmetry between them

Definition 9.1 (observations and layouts)**.** The observation universe is closed: a unit, a natural, an unsigned word, a boolean, a sum, an optional, a pair. No array, no structure of the state and no reference appears in it. The layout universe has three constructors: a stored observation, a binary product of layouts, and an array of a layout. The interpretation of a layout is the Lean type a program holds: a pair, or an `Array` of the interpretations of its element layout.

The asymmetry is the point. A layout may contain an array; an observation may not. The type an array of structures denotes is a pair of machine words × `Array` of those, and the type a read may return is built from atoms. Consequently a read cannot return a piece of the state, because no observation denotes one.

9.2 Paths, and a whole-array read that cannot be written down

Definition 9.2 (path and observation path)**.** A path into a layout is an address: the whole layout, a descent into the left or right component of a product, or a descent into an index of an array. The product steps exist only under a product and the array step only under an array, so the type of a path is indexed by the layout it walks. A function computed from the path gives the layout at its end. An *observation path* is a path together with the proof that its end is a stored observation.

The proof field in definition 9.2 is what makes the interface's guarantee structural rather than a rule. It is an equality $\mathsf{At}\ p = \texttt{.leaf}\ o$, and at an array or a product end that equality has no solution, so no observation path exists there. A whole-array read therefore cannot be *written down*, let alone performed, and the development states both sides of the same fact: a theorem that $\mathsf{At}\ \texttt{.here} \neq \texttt{.leaf}\ o$ at an array root, and a check that the elaborator rejects a use of the path that would have needed such an equality.

The one path step whose validity depends on a particular state is the array index, so the calculus asks separately whether a path addresses anything. That predicate is a pure traversal: it reads the elements along the path and never copies an array. Asking first is what lets the update itself be total, and what lets a failed write return the state it was given.

9.3 Reads, and the encoding the library owns

An observation path and a state yield the library's encoding of the observation at the path's end, or `none` when the path addresses nothing. The traversal that obtains the focus is private, so the raw focus is obtained and encoded in one step, and no function returns a state or a subtree.

The encoding has one clause per observation code, and no clause produces a location. From the clauses alone the development proves, for every code, that an encoded observation is injective and names no location, by an induction over the observation universe. The consequence is the read guarantee: a successful read names no `.loc`, so it cannot hand back a mutable part of the state. This is the fact that example 7.4 said an author could choose away; here no author chooses it, because the encoding is the library's.

9.4 Writes: a specification, an implementation, and a bridge

A write needs the state with the addressed part removed. That is the one-hole context of the path, which for this layout universe is its derivative in McBride's sense [7, 19]: at a leaf the hole is the whole value; at a product one component keeps the hole and the other is held whole; at an array the selected element is taken out and the array is held with a placeholder, together with the index that puts it back.

Definition 9.3 (the zipper as specification)**.** `open` takes a state apart into a context and a focus, or reports the original state for an index that addresses nothing at any depth; `plug` puts a focus back. Both are private, so a client cannot obtain a raw focus.

The zipper is the *specification*. The *implementation* is a structurally recursive update with one clause per path constructor, and the theorem that relates them is the bridge: on a state the private `open` takes apart, the recursion is the context `open` reported, plugged with the focus it reported. Everything else about writes follows from that bridge: reading back what was written, a failed write leaving the state alone, and a write at a path disjoint from a read path not changing what that read returns. Disjointness here excludes the equal paths and every ancestor/descendant pair, including two paths through the same array index, rather than merely paths whose final fields differ.

Two properties of the implementation are not about the zipper at all, and both are decisions taken because of the compiler.

- The new focus is threaded through the recursion as a *value* — the focus of the path — rather than as a function of the old state, so the write path allocates no closure.
- The recursion returns the state alone, not a state paired with the write's report. This is what lets the compiler's reuse pass rewrite a product cell rather than allocate a fresh one; with a pair return, the pass matched the transient pair instead of the stored cell, and the address changed. The paper returns to this in section 10, because it was found by measuring addresses and not by reading the source or the emitted C.

9.5 The array case is a take-out

A write below an array index cannot simply read the element and write it back: the array would still hold the element, so a write inside it would copy the element. The element must be *taken out*.

Definition 9.4 (take-out)**.** The array case exchanges the selected element for a placeholder, leaving a sibling array that keeps its size and every other slot, and hands the element back as the sole owner of it. The exchange and its inverse are private, like the zipper: they hand back the raw element that an update below the slot works on.

The array's exclusivity is not the element's. A take-out replaces one slot, and another slot — or a caller's own reference — may still point at the same element, so a write inside the element would copy it. The implementation therefore runs *two* exclusivity checks, and both are the difference between a mutation and a copy: one on the array, before the take-out, because a shared array is copied whole; and one on the extracted element, before the recursion below it runs, because a shared element is what a write inside it would copy. Each check names its site in the report.

Two remarks on the checks are worth making here, because they are the places where the discipline meets the compiler.

First, a pair is a second reference. The exchange returns the sibling array and the element together; while that pair is live, the element has two references, and checking it then fires on an element that is in fact uniquely owned. The implementation destructures the pair before anything is checked or recursed into, so the check sees one reference. That is a fact about the emitted code, and it was found by a harness case failing.

Second, the checks are the object interface's check from section 5, arrived at from the other direction: there, an instance supplied a check so that a copy would be loud; here, the library's own recursion installs the two checks its own writes need.

9.6 The sealed class

Definition 9.5 (sealed state variable)**.** An instance of `SealedStateVar` supplies two things: the layout its type is, and a proof $\texttt{rep} : \sigma = \mathsf{State}\ \textit{layout}$. It has no field for a read, a write or an encoding, so it cannot supply one. The library defines the operations once, from the equality: a read is the layout's read at the transported state, and a write is the layout's write with the state cast in and the state it returns cast out.

From the equality the library also derives a lawful state variable for the sealed type, by transporting its adapter along `rep`. The derived instance's read result type and its encoding are the library's by `rfl`: an instance has no field in which to put a different one, and the development proves the two equations rather than assuming them. The read, snapshot, frame and failure theorems are then restated for every sealed instance, each inherited by transport rather than proved per instance.

The comparison with example 7.4 is the point of the class. The lawful state-variable class still admits the array instance whose read is the state; the sealed class has no field to put that read in. And the guarantee is structural: a sealed read returns an encoding built from observations, which has no location branch, so it cannot hand back a mutable part of the state.

Example 9.6 (what the checks pin from outside)**.** The property that stands in for a proof of non-escape is that the raw routes cannot be written. From a module outside the library, the development pins, by checks whose elaboration must fail with a pinned message: that an instance adding an `observe` is rejected, that an instance adding an `encodeRead` is rejected, that `rep` cannot name a layout the type is not, that the `LawfulStateVar` fields `observe` and `encodeRead` are not names a sealed client can even write, and that the private pieces of the path calculus —

the context, its two directions, and the take-out and its inverse — are not reachable. Beside them are the affirmative controls: the class's fields are readable, the read and the write run on both shipped instances, and the two cast round trips hold.

Five rejections are pinned around the path calculus alone — the context, the open, the plug, the take-out and its inverse — which is how the raw-focus boundary is observed from outside the module rather than asserted.

The reader who wants the load-bearing detail can compare the two classes directly. A sealed instance's `rep` must be an equality and not an isomorphism, which is a real restriction and is stated as one: for a structure a user declares, the only relation to a layout's state type is an isomorphism, and the elaborator will not accept an equality. Such a type needs a layout that *names* it, which the current universe does not have, or a conversion on writes whose cost has not been measured. Arrays, nested arrays, pairs and the observation universe's leaves are usable, because the layout interpretation denotes them definitionally; a user structure is not.

10 Evidence, and the correspondence chain

Everything so far is a design with a proof obligation attached. This section states how the obligation is discharged in practice, where the evidence stops, and what has to be assumed beyond it.

10.1 What the harness measures

The run-time claims are established by an executable that observes addresses. For each case it records the address of the object before and after the operation; a copy allocates a new object, so an unchanged address is evidence of an update in place. The cases that ship are the following.

- *Unique writes.* The structured interface's write, and a structured program whose read phase returns a closure over its observations, each keep their array's address. For the layout calculus, four path writes — a left field, a right field, the last element and an out-of-bounds index — keep the buffer's address, keep the address of the product cell at the written index, and keep the sibling struct's address; the out-of-bounds case also returns the state unchanged.
- *Nested writes.* A write to a leaf of an inner array keeps the inner array's own address, which is the measurement that the take-out and the element's check are doing their work.
- *Controls that must copy.* A live second reference to the array, a live second reference to the focused inner array, a compile-time array literal, and a shared object in the counter example. Each asserts that the *address changes*.
- *Checked children.* Each unique case is run again in a child process with the in-place check armed, and must exit zero having reported that the check was enabled. The shared cases are run the same way and must *terminate* the process, naming the site. The armed control that must terminate is the counter's.

The last group is what turns the check from an assertion into an instrument. A unique case that exits zero with the check armed establishes that the check was in the write path and had nothing to report; a shared case that terminates establishes that the check fires when it should. Without the second, the first would be consistent with a check that never runs.

Two of the harness cases exist because an earlier version passed a reading of the source and failed at run time.

1. The take-out returns a pair. While that pair is live the element has two references, so the element's check fired on a uniquely owned element and the whole write copied. The fix is to

destructure the pair before checking or recursing; the emitted code then releases the pair and checks the focus with one reference.

2. With the update returning a state paired with its report, the reuse pass rewrote the transient pair rather than the stored cell, so the stored struct was fresh and copied — even though the emitted C contained the exclusivity test and the in-place field write. The fix is the state-only recursion of section 9. The address changed while the source and the C both looked right, which is the reason the instrument is an address assertion.

10.2 The shape rules

The measurements reduce to five rules that decide whether an update takes place, and it is worth writing them down, because four of them constrain whoever writes the update rather than the client.

1. The object was allocated at run time. A compile-time constant has count 0 and is never exclusive, which is why the session's array is created by the runner rather than by the caller.
2. Its count is 1 at the mutation: no alias live across the call, no projection of it taken and held, and no second use of the same value in the same expression.
3. A whole-value rebuild recycles the incoming cell when that cell is exclusive, but the pass matches on shape, so when more than one same-shaped constructor appears in the expression the cell can go to the wrong one.
4. A write nested inside a structure needs every node on the path to be exclusive at the moment of the write. A take-out makes the node owned; a borrowed accessor does not.
5. The write is a fixed-size element write. A store cannot grow an array.

Two of the five are settled by the *surface* rather than by care: the write installs an observation, so no client value — and hence no client alias — can be written into the state, and there is no growth operation. That is the sense in which the interface makes the losing shapes unwritable where types can say so. The other three are obligations on the library's own update paths, discharged by the shape of the recursion and checked by the harness.

It is also worth saying what a normal update looks like here, because the shape that copies is not the shape a write takes. A write is "check, then write", and the check returns a boolean and retains nothing, so the state's uses are sequential and the last is a move: the public write compiles to a bounds check and then the fused update, with no increment of the state at all. The copying shape — write and read one value at once — cannot be expressed through the interface, because a client has commands and not a state.

10.3 What is diagnostic

The in-place check is diagnostic and nothing more. It is the identity in the logic, so no specification or proof depends on it. It reads its switch once at process start, so a write costs a call, a load and a branch. In a run where the switch is off it neither proves nor reports anything; and in any run its verdict rests on the runtime's exclusivity primitive, which is part of the trusted base. The check's purpose is to make a copy loud when someone is looking for one. The address assertions, not the check, are what establish that an update took place.

10.4 The chain

The gap between the discipline and the compiled program is spanned by a chain of statements. Each link is either proved here, a property of the exported surface, measured on this toolchain,

or assumed. Naming them individually is the point: a single sentence such as "the discipline gives in-place update" would hide four distinct assumptions.

Link	Content	Status
1	The pure language's denotation of a program is its result	proved (section 3)
2	A store with count 1 is equivalent to the copy; with count ≥ 2 it is not	proved (theorem 3.6)
3	Every write site reaches its precondition, so $\mathsf{held} = 1$ there	proved (section 4)
4	$\mathsf{held} = 1$ at a write means no read share is outstanding	proved (the ledger and the primitives)
5	No client value names the session's state or a mutable part of it	structural (sections 6 and 9)
6	The library's own temporaries add no reference at a mutation	measured, per shape (section 10)
7	The compiler's counts follow the uses of the term, and its insertions implement them	assumed
8	The runtime's exclusivity test is $\mathrm{rc} = 1$, and the store primitive stores in place exactly when it holds	assumed
9	A compiled write updates in place on the shapes covered	measured (section 10)

Theorem 3.6 is what makes links 3 to 5 worth having at all. What they give is a source program whose every write site is reached with the object unshared by the interface — which is link 5's content — and with no reader outstanding, which is links 3 and 4. Links 7 and 8 are what carry that to the machine, and they are assumptions here rather than theorems: link 7 is the counting discipline documented by Ullrich and de Moura [29] and refined by Perceus [24], and link 8 is the runtime's contract. Link 6 is neither: it is a property of the library's own code, and it is where the paper's measurement effort went, because it is the only link under our control.

Corollary 10.1 (discipline adequacy, conditional)**.** *Assume links 6, 7 and 8. Let p be a program accepted by* `exec` *on a state that is uniquely owned when* `exec` *is called. Then every store site in p's compiled code is reached with the object's count one, and by corollary 3.7 every effect those stores perform is unobservable to a pure client.*

Remark 10.2 (what would make link 7 unconditional)**.** Link 7 could be replaced by a proof against a counted calculus, with the sealing assumptions as hypotheses: that the object never escapes, and that every read result is built from the observation universe. That turns "count one at a write" for a class of programs from measured into proved *in the model*, with the correspondence to compiled executions still assumed link by link. It is a different result from a measurement, and it should be labelled as one. The development here does not attempt it; the vendored counter model of appendix A is the vocabulary such a proof would be stated in, not the proof.

10.5 Non-claims

A list of what is not claimed is part of the result.

- No theorem here certifies that a compiled write updates in place, and none certifies anything about Lean's object graph. The in-place evidence is a measurement on named cases of one toolchain, and the emitted code is a description of that toolchain rather than a specification.
- The detachment laws are about the modelled encoding. `NoRef` says that an encoded read names no location; the sealed class removes the author's choice of that encoding but does not prove that the library's encoding matches the runtime's references.
- Sealing establishes non-escape through the exported operations. It says nothing about `unsafe` code, foreign functions, or values a client obtained outside the interface.

- The fork/join is notional. The executor is sequential, and a concurrent backend would still face the thread-shared marking of remark 6.1.
- A missed in-place update costs time and never correctness. Lean is pure, so every performance claim here is a claim about allocation.
- The counter-level facts inherited from the vendored model inherit that model's trust character, down to the generated certificates some of its proofs carry; the development prints, rather than assumes, which principles each of its own theorems rests on.

11 Related work

Fractional permissions and separation logic. Permissions as shares of an object are Boyland's [3], in the tradition of separation logic [23, 26]. Iris [9] and its proof-mode descendants [11], VeriFast [8] and Viper [20] reason this way about real memory. Our use of fractions is deliberately modest: no heap assertion language, no concurrent resource algebra, and no reasoning about interference between threads. What is borrowed is the arithmetic of shares and the observation that a write needs the whole. What is new is where the fractions live: in the type of a program, in a ledger the compiler erases, in a host language with no linear types.

Permissions with real mutation in a proof assistant. F⋆'s Steel [5] applies fractional permissions to arrays and extracts to C with real mutation, which makes it the closest existing system to the first half of this paper. Verus [12] combines Rust's own linear types with ghost permissions. Both are verification frameworks: the program is proved against a specification that mentions the heap. Ours is a discipline, not a logic: there is no specification of the object's contents, and the only property proved is the one the writer needs — authority, detachment, snapshots, frames. A reader who wants to state what a program computes would reach for a Hoare logic; the development's program type is the degenerate case in which the specification is the permission.

Hoare logic for state monads. The program type is a Hoare state monad in the style of Swierstra [28], who gives a Hoare logic for the state monad with preconditions and postconditions as type indices, and of Hoare Type Theory and Ynot [21, 22]. Our monad is a value of that shape with one addition: because the host language has no linear values, the precondition is a predicate on a ghost ledger rather than a separation-logic assertion, and the seal is a private capability rather than a linear hypothesis. The monadic presentation itself is standard [30].

Reference counting and in-place update. Lean's runtime is the reference-counting discipline of Ullrich and de Moura [29], with the reuse pass of Perceus [24], and Lean's own machine-checked model of the count arithmetic is the source of the counter vocabulary in appendix A. Koka's FP^2 [17] is the closest prior art on the other axis: it statically checks that a function runs *fully* in place. The comparison is instructive. FP^2 checks the property the compiler needs, by counting, for arbitrary recursive datatypes, and its check does not model sharing among readers. Ours models sharing, by construction, and gets one reference at the write by sealing the state away from clients; it is restricted to a layout universe of shapes the library can describe. The two are complementary: a counting discipline answers "does this function reuse everything", and a permission discipline answers "is this object unshared at this write".

Uniqueness and linearity in the host language. Clean's uniqueness types [1], Linear Haskell [2] and Rust's ownership discipline [10, 18] are the source-language answers to the same problem: make the count-one condition a typing condition. Lean has no such answer, which is what makes the interface-based discipline here necessary rather than merely optional. A reader

who has used Rust's borrows will recognise the shape of what is proved — a write needs exclusive access, reads may share — and will also recognise the difference: Rust's discipline is enforced by the type checker, ours by a ledger and a sealed interface, and ours therefore has to say where its enforcement stops.

Unobservable state. The pattern of a locally effectful computation observed only through a pure result is Launchbury and Peyton Jones's [13]: the state monad's thread is threaded and then hidden, so the effect cannot be observed. Our situation is different in the mechanism and the same in the spirit. There, the effect is invisible because the state is not shared; here, the effect is invisible because the compiler chooses to perform it exactly when no other reference exists, and the discipline's job is to make that choice reliable. Reynolds's syntactic control of interference [25] is the ancestor of the idea that a program's names, rather than its values, are what should be controlled; a token that is a name for a ledger entry and not a permission is a late echo of it.

Levitation and derivatives. The description universe is levitation [4]: describe, interpret, and prove once. The one-hole context of a path is the derivative of a data type [19], with the zipper of Huet [7] as the intuition. Our universe is non-recursive and much smaller than levitation's; what is carried over is the organisation — the description is data the library computes with, and the library, not the author, writes the operations.

Ghost state and erasure. The ledger is ghost state, in the same sense as Dafny's ghost declarations [15] and F⋆'s ghost effect [27], and the discipline depends on the compiler erasing arguments by type [6]. The measurements of what remains at run time — that a field-less structure compiles to a constant scalar, that arity reduction removes literal arguments, that a symbolic argument survives as an unused borrowed parameter — are properties of this toolchain's erasure, and they are the reason the discipline is free rather than merely cheap.

Trusted compilation. The last links of section 10's chain are the compiler and the runtime, which is the position any verified-by-measurement argument must take. CompCert [16] is the standard reference for what taking that chain seriously looks like; the development here takes the weaker option of naming the assumptions and measuring what it can.

12 Conclusion

A pure language can update in place because the update cannot be observed. The condition is that the updated object has one reference, and that condition is a fact about the compiled code rather than about the meaning of the program, so getting it where one wants it is a discipline rather than an inherent property of the source language.

This paper has followed that discipline from its premise to its generalisation. The premise was made precise in a small counted calculus, where the store rule at count one is proved equivalent to the copy and the store rule at higher counts is proved not to be. The discipline was given as fractional permissions in an erased ghost ledger, behind a program type that no client can build and that one entry point can run, which makes a write's authority a proof obligation and costs nothing at run time. It was then lifted three times: to any object, by letting the object carry its own check; to any state type, by turning the logical part of the object's rules into proof fields and stating the model's theorems once; and to any shape, by describing shapes in a universe and transporting the universe's one instance along a bijection to the author's type. Finally, the route by which an author could still hand back the state was closed, by a class in which the library owns the reading, the writing and the encoding, and by a path calculus in which a whole-array read cannot even be written down.

Two things are, we think, the parts worth keeping. The first is the shape of the result. Nothing here is a new type system and nothing here changes Lean. The discipline's enforcement is spread over four mechanisms, each doing what it is good at: proofs for the accounting, private constructors and a capability for sealing, an equality between a type and a described layout for the meaning of a read, and a check that is the identity in the logic for making a copy loud at run time. The second is the honest accounting. The paper's chain of evidence has nine links, of which five are proved or structural, two are measured, and two are assumed; knowing which are which is most of the work of trusting any of it.

The obvious extensions are the ones the design already points at: a recursive description universe, so that lists and trees are describable and not only arrays and products; layouts that *name* a user's structure, so that the equality the sealed class needs can be given for declared types rather than only for types the layout interpretation denotes definitionally; per-path permissions, so that two writers can hold disjoint parts of one object; and a mutable cell behind the sealed interface, which is what a real state variable would want and which the current development approximates with a logical heap. A concurrent backend is the largest of them, and remark 6.1 is the note on why it is a runtime problem as much as a permission problem.

A Quoted declarations

The listings below are read from the library by `\lstinputlisting` with an explicit line range, so the paper quotes the source and not a transcription of it. Each is preceded by a `%!check:` directive naming a fragment that the quoted range must contain; `check-listings.py` fails if a range has drifted. This checks that the paper is quoting what it claims to quote, and not that the Lean compiles: the library is built separately.

The ledger and the held fraction *(Scidonia/FracPerm/Ledger.lean, lines 16–22)*

```
abbrev Ledger := List (Nat × Rat)

namespace Ledger

/-- The fraction of the array the caller still holds: the whole minus what is
lent out. The empty ledger holds everything, so writes are allowed there. -/
def held (l : Ledger) : Rat := 1 - (l.map Prod.snd).sum
```

Returning a read gives back exactly its fraction *(Scidonia/FracPerm/Ledger.lean, lines 32–37)*

```
held fraction by exactly ‘q‘. This is the whole point of the interface: a
returned read gives back exactly what it took, no more and no less. -/
theorem held_erase_of_mem {n : Nat} {q : Rat} {l : Ledger} (h : (n, q) ∈ l) :
    held (l.erase (n, q)) = held l + q := by
  induction l with
  | nil => simp at h
```

The object interface *(Scidonia/FracPerm/Hoare.lean, lines 107–132)*

```
class FracObject (σ : Type) (ReadOp : outParam Type) (WriteOp : outParam Type) where
  /-- The value a read operation yields. -/
  Result : ReadOp → Type
  /-- Perform a read operation. -/
  observe : (o : ReadOp) → σ → Result o
  /-- Perform a write operation. -/
  update : WriteOp → σ → σ
  /-- The check ‘write‘ runs before ‘update‘, given the site, the operation
  about to run and the state it is about to run on.
```

```
  It is the identity in the logic, so no specification or proof mentions it; a
  law (‘check_eq‘) rather than a definitional equation, because an instance is
  free to define it as compilation-only (an identity definition with an
  ‘implemented_by‘ implementation), and its compiled form is what makes a
  copying update loud.

  It defaults to ‘assertExclusive‘ on the whole object. An object whose mutable
  state is a field of a larger object -- a structure with an array field, say --
  replaces the default with a check of that field, so that the copy the field
  would take is caught; the operation is a parameter so that such a check can
  leave alone what the operation does not write. -/
  check : String → WriteOp → σ → σ := fun site _ s => assertExclusive site s
  /-- ‘check‘ is the identity: ‘check site o s = s‘. An instance whose ‘check‘
  is an identity definition with an ‘implemented_by‘ implementation proves this
  by ‘rfl‘. -/
  check_eq : ∀ site o s, check site o s = s
```

The program type *(Scidonia/FracPerm/Hoare.lean, lines 178–184)*

```
structure HSt (σ : Type) (P : Pre σ) (β : Type) (Q : Post σ β) where
  private mk ::
  /-- Run the program on a state, given a proof that some ledger meets the
  precondition. Private to this module, and usable only with a ‘Cap‘:
  ‘exec‘ is the entry point. -/
  private run : Cap → (s : σ) → (∃ l, P l s) →
    { r : β × σ // ∀ l, P l s → ∃ l', Q l s r.1 l' r.2 }
```

The primitives and their specifications *(Scidonia/FracPerm/Hoare.lean, lines 229–266)*

```
the fraction are ordinary arguments; see the module note for what the compiler
does with them. -/
def acquire (n : Nat) (q : Rat) :
    HSt σ (fun l _ => n ∉ l.map Prod.fst ∧ 0 < q ∧ q ≤ l.held) (ReadTok n q)
      (fun l s _ l' s' => l' = (n, q) :: l ∧ s' = s) :=
  ⟨fun _ s _ => ⟨(⟨⟩, s), fun _ _ => ⟨_, rfl, rfl⟩⟩⟩

/-- Read operation ‘o‘, yielding the value ‘observe o s‘. Only a holder of a
token whose ledger entry is still present may do so; reading changes
nothing. -/
def read [FracObject σ ReadOp WriteOp] {n : Nat} {q : Rat} (_t : ReadTok n q) (o : ReadOp) :
    HSt σ (fun l _ => (n, q) ∈ l) (FracObject.Result (σ := σ) o)
      (fun l s v l' s' => l' = l ∧ s' = s ∧ v = FracObject.observe o s) :=
  ⟨fun _ s _ => ⟨(FracObject.observe o s, s), fun l _ => ⟨l, rfl, rfl, rfl⟩⟩⟩

/-- Give the fraction ‘q‘ back and consume the token's ledger entry. A second
‘release‘ of the same token is rejected: the entry is gone, and a ‘read‘ with
it is rejected for the same reason. -/
def release {n : Nat} {q : Rat} (_t : ReadTok n q) :
    HSt σ (fun l _ => (n, q) ∈ l) Unit
      (fun l s _ l' s' => l' = l.erase (n, q) ∧ s' = s) :=
  ⟨fun _ s _ => ⟨((), s), fun _ _ => ⟨_, rfl, rfl⟩⟩⟩

/-- Write with operation ‘o‘. Requires the caller to hold the whole state: some
entry is outstanding in every other case.

Before the update, the object's ‘check‘ runs on the operation and the state.
It is the identity in the logic -- ‘check_eq‘ is what lets the postcondition
speak of ‘update o s‘ alone -- and its compiled form is the in-place check; see
‘Scidonia.FracPerm.InPlace‘ and ‘FracObject.check‘. -/
def write [FracObject σ ReadOp WriteOp] (o : WriteOp) :
    HSt σ (fun l _ => l.held = 1) Unit
```

```
      (fun l s _ l' s' => l' = l ∧ s' = FracObject.update o s) :=
  ⟨fun _ s _ =>
      ⟨((), FracObject.update o (FracObject.check "Scidonia.FracPerm.HSt.write" o s)),
        fun l _ => ⟨l, rfl,
          congrArg (FracObject.update o) (FracObject.check_eq _ o s)⟩⟩⟩
```

The lawful state variable *(Scidonia/FracPerm/StateVar/Class.lean, lines 59–91)*

```
class LawfulStateVar (σ : Type) (Init : outParam Type)
    (ReadOp : outParam Type) (WriteOp : outParam Type) where
  /-- The value a read operation yields. A read operation may yield a value of
  any type, one per operation, as `FracObject.Result` does. -/
  Result : ReadOp → Type
  /-- A state built from an initializer. -/
  initial : Init → σ
  /-- The admissible states of the variable. -/
  Valid : σ → Prop
  /-- Perform a read operation. It is a pure function of the operation and the
  state. -/
  observe : (o : ReadOp) → σ → Result o
  /-- Replace the state by an exclusive logical replacement, and report whether
  the write took effect (`false` for a write that the variable declined, as an
  out-of-bounds array write is). -/
  replace : WriteOp → σ → σ × Bool
  /-- Encode a read result as a logical value. -/
  encodeRead : (o : ReadOp) → Result o → Value
  /-- `initial` builds admissible states: fresh construction is admissible. -/
  initial_valid : ∀ i : Init, Valid (initial i)
  /-- `replace` closes admissible states: an exclusive logical replacement of an
  admissible state is admissible. -/
  replace_valid : ∀ (w : WriteOp) (s : σ),
    Valid s → Valid (replace w s).1
  /-- `encodeRead` distinguishes read results: two results an operation encodes
  as the same value are the same result. -/
  encodeRead_injective : ∀ (o : ReadOp) (x y : Result o),
    encodeRead o x = encodeRead o y → x = y
  /-- A primitive observation of a valid state is detached from every root: its
  encoding names no `.loc ℓ`, for every `ℓ`, beneath any compound
  observation. -/
  observe_detached : ∀ (ℓ : Loc) (o : ReadOp) (s : σ),
    Valid s → NoRef ℓ (encodeRead o (observe o s))
```

The description universe *(Scidonia/FracPerm/StateVar/Description.lean, lines 24–46)*

```
inductive Desc where
  /-- The unit type. -/
  | unit
  /-- The type of naturals. -/
  | nat
  /-- The sum of two described types. -/
  | sum (left right : Desc)
deriving DecidableEq, Repr

/-- The type a description denotes: `Unit`, `Nat`, or a `Sum` of the two
sub-descriptions' types.

`El` is an `abbrev`, so `El slotDesc` is reducible for a client: `Sum Unit Nat`
is its definitional form, and a literal such as `Sum.inr 7` can be given at an
`El` type without a type ascription. -/
abbrev El : Desc → Type
  | .unit => Unit
  | .nat => Nat
```

```
  | .sum left right => Sum (El left) (El right)

/-- The description of a slot: a unit (the empty slot) summed with a natural
(the payload of a full slot). -/
def slotDesc : Desc := .sum .unit .nat
```

The sealed class *(Scidonia/FracPerm/StateVar/Sealed.lean, lines 52–60)*

```
a layout. An instance supplies the layout and the equality; the library defines
the read, the write and the encoding from them, so an instance author cannot
choose any of the three. -/
class SealedStateVar (σ : Type) where
  /-- The layout whose state type ‘σ‘ is, up to ‘rep‘. -/
  layout : Layout
  /-- ‘σ‘ is the layout's state type, up to this equality. -/
  rep : σ = State layout
```

The library's read of a sealed state *(Scidonia/FracPerm/StateVar/Sealed.lean, lines 66–74)*

```
  cast (SealedStateVar.rep (σ := σ)) s

/-- The library's read of a sealed state: the state transported to its layout by
‘rep‘, read by ‘Layout5.read?‘. The result is an ‘Option Value‘ from the closed
observation universe, never a state or a subtree. -/
def read? {σ : Type} [SealedStateVar σ]
    (p : ObsPath (SealedStateVar.layout (σ := σ))) : σ → Option Value :=
  fun s => Layout5.read? p (toState s)
```

The library's write of a sealed state *(Scidonia/FracPerm/StateVar/Sealed.lean, lines 83–90)*

```
The write's report is ‘Layout5.setLeaf‘'s: ‘false‘ for a path that addresses
nothing, in which case the state is unchanged. -/
def setLeaf {σ : Type} [SealedStateVar σ]
    (p : ObsPath (SealedStateVar.layout (σ := σ))) (x : ObsEl p.obs) : σ → σ × Bool :=
  fun s =>
    let r := Layout5.setLeaf p x (toState s)
    (ofState r.1, r.2)
```

The observation universe *(Scidonia/FracPerm/StateVar/Layout5.lean, lines 91–119)*

```
inductive ObsDesc where
  /-- The unit type. -/
  | unit
  /-- The type of naturals. -/
  | nat
  /-- The type of unsigned machine words. -/
  | uint
  /-- The type of booleans. -/
  | bool
  /-- A sum of two described types. -/
  | sum (left right : ObsDesc)
  /-- An optional described type. -/
  | option (elem : ObsDesc)
  /-- A pair of two described types. -/
  | pair (left right : ObsDesc)
deriving DecidableEq, Repr

/-- The type a leaf description denotes. Units, naturals, words and booleans
are immutable data; a sum, option or pair is built from them. No case is an
array, a struct or a reference. -/
abbrev ObsEl : ObsDesc → Type
```

```
  | .unit => Unit
  | .nat => Nat
  | .uint => UInt64
  | .bool => Bool
  | .sum a b => Sum (ObsEl a) (ObsEl b)
  | .option a => Option (ObsEl a)
  | .pair a b => ObsEl a × ObsEl b
```

The layout universe and its interpretation *(Scidonia/FracPerm/StateVar/Layout5.lean, lines 122–138)*

```
inductive Layout where
  /-- A leaf of the observation universe. -/
  | leaf (o : ObsDesc)
  /-- A binary product of two layouts. -/
  | struct (left right : Layout)
  /-- An array of a layout. -/
  | arr (elem : Layout)
deriving DecidableEq, Repr

/-- The state type a layout denotes: an element of the observation universe, a
pair of states, or an array of states. -/
abbrev State : Layout → Type
  | .leaf o => ObsEl o
  | .struct a b => State a × State b
  | .arr a => Array (State a)

/-! ## Paths
```

Typed paths and the layout at their end *(Scidonia/FracPerm/StateVar/Layout5.lean, lines 147–166)*

```
inductive Path : Layout → Type where
  /-- The path to the whole layout. -/
  | here {d : Layout} : Path d
  /-- Descend into the left component of a struct. -/
  | left {a b : Layout} (p : Path a) : Path (.struct a b)
  /-- Descend into the right component of a struct. -/
  | right {a b : Layout} (p : Path b) : Path (.struct a b)
  /-- Descend into element `i` of an array. -/
  | index {a : Layout} (i : Nat) (p : Path a) : Path (.arr a)
deriving DecidableEq

/-- The layout at the end of a path. -/
abbrev At {d : Layout} : Path d → Layout
  | .here => d
  | .left p => At p
  | .right p => At p
  | .index _ p => At p

/-- A read path: a path together with the leaf description its end must have.
The proof field is why no `ObsPath` exists for a `.struct` or `.arr` root --
```

The observable path *(Scidonia/FracPerm/StateVar/Layout5.lean, lines 169–177)*

```
structure ObsPath (d : Layout) where
  /-- The path. -/
  path : Path d
  /-- The leaf description at the path's end. -/
  obs : ObsDesc
  /-- The end of the path is that leaf. -/
```

```
  leaf : At path = .leaf obs

/-- Concatenate two paths: the steps of the first, then the steps of the second
```

Disjointness of paths *(Scidonia/FracPerm/StateVar/Layout5.lean, lines 203–208)*

```
anything. This excludes the equal paths and every ancestor/descendant pair,
including two paths through the same array index. -/
def Disjoint {d : Layout} (p q : Path d) : Prop :=
  (∀ r : Path (At p), q ≠ Path.append p r) ∧ (∀ r : Path (At q), p ≠ Path.append q r)

/-- Nothing is disjoint from the root path, so a write at the root reaches every
```

The take-out *(Scidonia/FracPerm/StateVar/Layout5.lean, lines 298–308)*

```
/-- Take the element at `i` out of an array, leaving `d` in its place: the array
keeps its size and every other slot. When the array is exclusively owned the
exchange is an update in place and hands the element back; the array's
exclusivity is not the element's, though -- the take-out replaces one slot, and
another slot may still point at the same object -- so the caller checks the
extracted element before writing inside it. Without the array's exclusivity the
exchange copies the array, and the array the caller kept still holds the
element. -/
private def takeOut (s : Array α) (i : Nat) (h : i < s.size) (d : α) : Array α × α :=
  let r := s.swapAt i d h
  (r.2, r.1)
```

The library's encoding of an observation *(Scidonia/FracPerm/StateVar/Layout5.lean, lines 457–471)*

```
boolean use the corresponding `Value` constructor; sum, option and pair recurse,
so a `.loc` can never appear. -/
def encodeObs : (o : ObsDesc) → ObsEl o → Value
  | .unit, _ => .unit
  | .nat, n => .nat n
  | .uint, n => .uint n
  | .bool, b => .bool b
  | .sum a _, Sum.inl x => .inl (encodeObs a x)
  | .sum _ b, Sum.inr y => .inr (encodeObs b y)
  | .option _, none => .none
  | .option a, some x => .some (encodeObs a x)
  | .pair a b, (x, y) => .pair (encodeObs a x) (encodeObs b y)

/-- `encodeObs` distinguishes observations of one description: two elements it
encodes as the same value are the same element. The proof is a structural
```

The one-hole context *(Scidonia/FracPerm/StateVar/Layout5.lean, lines 539–555)*

```
/-- The one-hole context of a path: the parts of a state other than the focus.
The array case keeps the array the take-out left and the focus's index. -/
private inductive Context : (d : Layout) → Path d → Type where
  /-- The whole state is the focus. -/
  | here (d : Layout) : Context d .here
  /-- The left component is inside; the right is held. -/
  | left {a b : Layout} {p : Path a} (inner : Context a p) (right : State b) :
      Context (.struct a b) (.left p)
  /-- The right component is inside; the left is held. -/
  | right {a b : Layout} {p : Path b} (left : State a) (inner : Context b p) :
      Context (.struct a b) (.right p)
  /-- Element `i` is inside; the array the take-out left is held, and
```

```
  `i < siblings.size` is what lets `plug` land the focus back at the index. -/
  | index {a : Layout} {p : Path a} (i : Nat) (siblings : Array (State a))
      (bound : i < siblings.size) (inner : Context a p) : Context (.arr a) (.index i p)

/-- Fill a one-hole context with a focus: the state the context describes, with
```

The library's read *(Scidonia/FracPerm/StateVar/Layout5.lean, lines 679–684)*

```
end, or `none` when the path does not address anything. The result is an
immutable `Value`; there is no route from here to the state or a subtree. -/
def read? {d : Layout} (p : ObsPath d) (s : State d) : Option Value :=
  (leafAt? p.path p.obs p.leaf s).map (encodeObs p.obs)

/-! ## Validity and the fused update
```

The fused update *(Scidonia/FracPerm/StateVar/Layout5.lean, lines 710–729)*

```
the recursion as the focus of the path rather than as a function of the old one,
so the write path allocates no closure. The result is the state alone, so a
struct cell can be reused for the struct the recursion rebuilds -- which a
`State d × Bool` result would prevent, because the pair the compiler then reuses
is the transient one. -/
private def setAt : {d : Layout} → (p : Path d) → State (At p) → State d → State d
  | _, .here, y, _ => y
  | _, .left q, y, (u, v) => (setAt q y u, v)
  | _, .right q, y, (u, v) => (u, setAt q y v)
  | _, .index i q, y, s =>
      if h : i < s.size then
        match takeOutAt (assertExclusiveArray openSite s) i h (stateDefault _) with
        | ⟨(sib, focus), hbound⟩ =>
          plugAt sib i hbound (setAt q y (assertExclusive structSite focus))
      else s

/-- `inBounds` at the root path: the root addresses the state. -/
@[simp] private theorem inBounds_here {d : Layout} (s : State d) :
    inBounds (Path.here : Path d) s = true := rfl
```

The public write *(Scidonia/FracPerm/StateVar/Layout5.lean, lines 772–784)*

```
the new observation, and whether the write took effect (`false` for an
out-of-bounds index, in which case the state is unchanged). `inBounds` decides
first, so the fused update runs only on a path that addresses something; each
struct cell is then rebuilt from its updated component, and an array element is
taken out, checked, updated and put back. The new focus is cast once, here, and
threaded through the update as a value. -/
def setLeaf {d : Layout} (p : ObsPath d) (x : ObsEl p.obs) (s : State d) :
    State d × Bool :=
  if inBounds p.path s then
    (setAt p.path (cast (congrArg State p.leaf.symm) x) s, true)
  else (s, false)

/-! ## The zipper's round trip
```

What a write tests *(Scidonia/FracPerm/StateVar/RefCount.lean, lines 713–728)*

```
store in place: the count is single-threaded and stands for exactly one
reference.

{name}`isSt` alone is not enough. It admits a count that stands for many
references and, once the count has overflowed while single-threaded, one that
stands for no reference this count can still track; either way a second
```

```
reference exists that a store in place would lose. -/
def IsExclusive (rc : Int32) : Prop := isSt rc ∧ rc = 1

/-- The runtime's {lit}`is_unshared`, whether one thread owns the object. It is
strictly weaker than {name}`IsExclusive`: {name}`isUnshared` is
{lit}`rc > 0 || rc ≤ LEAN_RC_STUCK_ST`, so a count that overflowed while
single-threaded is unshared while standing for far more than one reference. A
write that tests {name}`IsUnshared` and then stores in place drops the other
references; the two tests are not interchangeable. -/
def IsUnshared (rc : Int32) : Prop := isUnshared rc
```

References

[1] E. Barendsen and S. Smetsers. Conventional and uniqueness typing in graph rewrite systems. In *FSTTCS*, 1993.

[2] J.-P. Bernardy, M. Boespflug, R. R. Newton, S. Peyton Jones, and A. Spiwack. Linear Haskell: practical linearity in a higher-order polymorphic language. *POPL*, 2018.

[3] J. Boyland. Checking interference with fractional permissions. In *Static Analysis (SAS)*, LNCS 2694, pages 55–72, 2003. DOI: 10.1007/3-540-44898-5_4.

[4] J. Chapman, P.-E. Dagand, C. McBride, and P. Morris. The gentle art of levitation. In *ICFP*, pages 3–14, 2010. DOI: 10.1145/1863543.1863547.

[5] A. Fromherz, A. Rastogi, N. Swamy, S. Gibson, G. Martínez, D. Merigoux, and T. Ramananandro. Steel: proof-oriented programming in a dependently typed concurrent separation logic. In *ICFP*, 2021. DOI: 10.1145/3473590.

[6] G. Gilbert, J. Cockx, M. Sozeau, and N. Tabareau. Definitional proof-irrelevance without K. *POPL*, 2019.

[7] G. Huet. The zipper. *Journal of Functional Programming*, 7(5):549–554, 1997.

[8] B. Jacobs, J. Smans, P. Philippaerts, F. Vogels, W. Penninckx, and F. Piessens. VeriFast: a powerful, sound, predictable, fast verifier for C and Java. In *NASA Formal Methods*, 2011.

[9] R. Jung, D. Swasey, F. Sieczkowski, K. Svendsen, A. Turon, L. Birkedal, and D. Dreyer. Iris: monoids and invariants as an orthogonal basis for concurrent reasoning. *POPL*, 2015.

[10] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. RustBelt: securing the foundations of the Rust programming language. *POPL*, 2018.

[11] R. Krebbers, A. Timany, and L. Birkedal. The essence of higher-order concurrent separation logic. In *ESOP*, 2017.

[12] A. Lattuada, T. Hance, C. Cho, M. Brun, I. Subasinghe, Y. Zhou, J. Howell, B. Parno, and C. Hawblitzel. Verus: verifying Rust programs using linear ghost types. *OOPSLA*, 2023. DOI: 10.1145/3586037.

[13] J. Launchbury and S. Peyton Jones. State in Haskell. *Lisp and Symbolic Computation*, 8(4):293–341, 1995.

[14] Lean FRO. `tests/elab/rc_model.lean`, Lean 4, tag `v4.35.0-rc3`. A machine-checked model of `lean_inc_ref_n` and `lean_dec_ref`, vendored into `Scidonia/FracPerm/StateVar/RefCount.lean`.

[15] K. R. M. Leino. Dafny: an automatic program verifier for functional correctness. In *LPAR*, 2010.

[16] X. Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.

[17] A. Lorenzen, D. Leijen, and W. Swierstra. FP^2: fully in-place functional programming. *PACMPL*, 7(ICFP), 2023. DOI: 10.1145/3607840.

[18] N. D. Matsakis and F. S. Klock. The Rust language. *ACM SIGAda Ada Letters*, 34(3), 2014.

[19] C. McBride. The derivative of a regular type is its type of one-hole contexts. Extended abstract, 2001.

[20] P. Müller, M. Schwerhoff, and A. J. Summers. Viper: a verification infrastructure for permission-based reasoning. In *VMCAI*, 2016.

[21] A. Nanevski, G. Morrisett, and L. Birkedal. Hoare type theory, polymorphism and separation. *Journal of Functional Programming*, 18(5–6), 2008.

[22] A. Nanevski, G. Morrisett, A. Shinnar, P. Govereau, and L. Birkedal. Ynot: dependent types for imperative programs. In *ICFP*, 2008.

[23] P. W. O'Hearn, J. C. Reynolds, and H. Yang. Local reasoning about programs that alter data structures. In *Computer Science Logic (CSL)*, 2001.

[24] A. Reinking, N. Xie, L. de Moura, and D. Leijen. Perceus: garbage free reference counting with reuse. In *PLDI*, 2021. DOI: 10.1145/3453483.3454032.

[25] J. C. Reynolds. Syntactic control of interference. In *POPL*, 1978.

[26] J. C. Reynolds. Separation logic: a logic for shared mutable data structures. In *LICS*, 2002.

[27] N. Swamy, C. Hriţcu, C. Keller, A. Rastogi, A. Delignat-Lavaud, S. Forest, K. Bhargavan, C. Fournet, P.-Y. Strub, M. Kohlweiss, J.-K. Zinzindohoué, and S. Zanella-Béguelin. Dependent types and multi-monadic effects in F*. In *POPL*, 2016.

[28] W. Swierstra. A Hoare logic for the state monad. In *Theorem Proving in Higher Order Logics (TPHOLs)*, 2009.

[29] S. Ullrich and L. de Moura. Counting immutable beans: reference counting optimized for purely functional programming. In *Implementation and Application of Functional Languages (IFL)*, 2019. DOI: 10.1145/3412932.3412935.

[30] P. Wadler. The essence of functional programming. In *POPL*, 1992.